

Trabalho Final da Disciplina de PAA - Sudoku: Problema e algoritmos de resolução

Antonio Herik Cosmo Martins¹, Bruno Lima Ferreira²,
Daniel Almeida de Freitas³, Victor Martins Vieira⁴

¹Campus Quixadá – Universidade Federal do Ceará (UFC)
Caixa Postal 63902-580 – 91.501-970 – Quixadá– CE – Brazil

²Projeto de Análise de Algoritmos

{herikcosmo, danielaalmeida, victormartinsbr, brunoufc}@alu.ufc.br

Abstract. *This project was proposed as the final assignment for the course "Algorithm Design and Analysis" in the Software Engineering program at UFC-Quixadá. The following work presents the problem of solving the combinatorial puzzle "Sudoku," along with the presentation of two algorithms that solve it, proof of correctness, input and output examples, and a comparison of execution times. The selected algorithms were brute-force backtracking and Algorithm SaturBFS.*

Resumo. *Este trabalho foi proposto como trabalho final da disciplina de Projeto e Análise de Algoritmos do curso de Engenharia de Software da UFC-Quixadá. O trabalho a seguir consiste na apresentação do problema de resolução do quebra-cabeças combinatório "Sudoku", além da apresentação de dois algoritmos que o resolvem, prova de corretude, exemplos de entradas e saídas, e comparação dos tempos de execução. Os algoritmos selecionados foram o de backtracking com força bruta e o Algoritmo SaturBFS.*

1. Introdução

Inventado em 1979, o Sudoku é um jogo de quebra-cabeça combinatório baseado em lógica, popular entre pessoas de todas as idades, no qual [Delahaye 2006], ganhou rapidamente popularidade mundial. Ele se destaca como um passatempo que desafia os jogadores a exercitar suas habilidades de lógica, dedução e resolução de problemas. O jogo atrai tanto jogadores em busca de entretenimento, quanto estudiosos interessados em explorar questões matemáticas e computacionais relacionadas ao jogo [Oliveira 2024].

1.1. Motivação

Resolver o Sudoku por meios algorítmicos é uma tarefa interessante não apenas por sua relevância nos campos da matemática e teoria dos jogos, mas também porque desafia o desenvolvimento de técnicas que podem ser aplicadas em áreas mais amplas da ciência da computação e inteligência artificial. Atualmente, o Sudoku não é apenas um jogo, mas também é considerado um problema central em muitas aplicações do mundo real, na vida diária e na engenharia industrial [Wang et al. 2024], usado como por exemplo na criptografia de dados [Jana et al. 2019], em tecnologias de ocultação de dados em imagens

[Li et al. 2021] e também em sistemas fotovoltaicos [Horoufiany and Ghandehari 2018]. Dito isso, O desenvolvimento de algoritmos para resolvê-lo transcende a pesquisa acadêmica, tornando-se uma ferramenta valiosa para aplicações do mundo real, o que faz com que ele atraia cada vez mais atenção [Wang et al. 2024].

2. Definindo o Problema

Dado um tabuleiro quadrado de dimensão $n \times n$, onde n é um número perfeito ao quadrado ($n = k^2$, para algum inteiro positivo k), o problema do Sudoku consiste em preencher todas as **células** desse tabuleiro com números inteiros de 1 a n . Essas células estão organizadas em n **linhas**, n **colunas** e n **blocos** de dimensão $k \times k$. O desafio é garantir que cada número entre 1 e n apareça exatamente uma vez em cada linha, coluna e bloco, respeitando as condições impostas pelas células previamente preenchidas no início do jogo. A 1 Figura 1 retirada do trabalho de [Takano et al. 2015] apresenta uma instancia do Sudoku no tamanho clássico 9×9 e uma possível solução.

4					8		5	
	3							
			7					
	2						6	
				8		4		
				1				
			6		3		7	
5			2					
1		4						

(a)

4	1	7	3	6	9	8	2	5
6	3	2	1	5	8	9	4	7
9	5	8	7	2	4	3	1	6
8	2	5	4	3	7	1	6	9
7	9	1	5	8	6	4	3	2
3	4	6	9	1	2	7	5	8
2	8	9	6	4	3	5	7	1
5	7	3	2	9	1	6	8	4
1	6	4	8	7	5	2	9	3

(b)

Figure 1. Exemplo de jogo clássico de Sudoku. (a) Instância com 17 casas inicialmente preenchidas. (b) Solução.

O Sudoku oferece grande flexibilidade na escolha dos símbolos e em outros elementos do jogo, permitindo o surgimento de várias variantes. Neste trabalho, focaremos na resolução de sudokus tradicionais, que seguem as regras descritas em 2. Ademais, além dos conceitos de célula, linha, coluna e bloco apresentado acima, utilizaremos neste trabalho a terminologia abaixo apresentada por [Takano et al. 2015].

- **Grid:** um tabuleiro $n \times n$ do jogo.
- **Valor:** Um inteiro entre 1 a n
- **Candidato:** Todo valor que, em dado momento, pode ser inserido em uma determinada célula (não violando as condições do problema).
- **Unidade:** Uma referência geral para linha, coluna ou bloco.
- **Grid incompleta:** Uma é uma solução parcial do Sudoku, com algumas células preenchidas, mas não todas.
- **Grid completa:** Uma solução final do Sudoku, com todas as células devidamente preenchidas.

Em relação à resolução, existem diversos algoritmos que utilizam diferentes técnicas e heurísticas para resolver o Sudoku. No entanto, o Sudoku é um problema NP-difícil [Yato and Seta 2003], e os algoritmos conhecidos possuem um tempo de execução

exponencial em n . Dito isso, focaremos em instâncias menores, como 9×9 e outras. Na Subseção 2.1, serão apresentadas brevemente algumas técnicas existentes e quais delas iremos utilizar, bem como os algoritmos que implementam essas técnicas escolhidas neste trabalho.

2.1. Técnicas de Resolução do Sudoku e Algoritmos que serão utilizados

Neste trabalho, utilizaremos duas técnicas para resolver o problema do Sudoku: **Backtracking** e **SaturBFS (Saturated Breadth-First Search)**. Essas técnicas representam abordagens distintas para resolver o quebra-cabeça, oferecendo diferentes vantagens em termos de eficiência, clareza de implementação e capacidade de encontrar soluções em tempo hábil.

2.1.1. Backtracking

O **Backtracking** é uma técnica clássica de busca exaustiva utilizada para resolver problemas de busca e otimização combinatória, como o Sudoku. Este método explora todas as combinações possíveis para preencher a grid do Sudoku, verificando se cada valor inserido em uma célula não viola as restrições do jogo (ou seja, não há repetições de números em linhas, colunas ou blocos).

2.1.2. SaturBFS (Saturated Breadth-First Search)

O **SaturBFS** é um algoritmo baseado em coloração de grafos e busca em largura, otimizado para resolver o Sudoku de forma mais eficiente, especialmente em instâncias de maior complexidade. Este algoritmo é uma combinação das técnicas de **Busca em Largura (BFS)** e do algoritmo **DSatur**, utilizado em problemas de coloração de grafos, que prioriza os vértices com maior saturação (isto é, aqueles que têm mais vizinhos já coloridos).

3. Propriedades de Entrada e Saída

3.1. Entradas

As entradas devem conter as células iniciais preenchidas para seguir as regras para a formação de um sudoku válido, conforme descrito em 2. Além disso, as entradas deverão conter pelo menos 17 células já preenchidas, devido à grande dificuldade de produzir sudokus com menos de 17 células, conhecida como o “problema do sudoku mínimo” [Lin and Wu 2011].

De modo geral, uma métrica para o nível de dificuldade do sudoku, especialmente em termos de percepção humana, é a quantidade de células pré-preenchidas. É importante destacar que há diferentes abordagens para a definição dos *níveis de dificuldade* do sudoku, de modo que, em certos casos, apenas a quantidade de células preenchidas não é suficiente. Assim, como forma de simplificação, utilizaremos entradas pré-produzidas e classificadas em três níveis de dificuldade (**Fácil**, **Médio** e **Difícil**) pelo site SudokuTodo¹.

¹Site: <https://sudokutodo.com/generator>

Ademais, para progredir o tamanho da entrada, usaremos sudokus com as grades nas seguintes configurações: (4×4) , (6×6) , (8×8) , (9×9) , (10×10) , (12×12) .

1			2
4			
		1	3
3			

(a)

1	3	4	2
4	2	3	1
2	4	1	3
3	1	2	4

(b)

Figure 2. Exemplo de Sudoku (4×4) classificado com o nível de dificuldade Fácil. (a) Instância parcialmente preenchida. (b) Solução.

8				3	2			
	1			9			4	8
9			8	6			1	3
		6		7		4	8	
				8				1
							7	
				4	9		2	5
3			5			9		
	2			7				

(a)

8	6	4	1	3	2	7	5	9
2	1	3	7	9	5	6	4	8
9	7	5	8	6	4	2	1	3
1	5	6	9	7	3	4	8	2
7	3	2	4	8	6	5	9	1
4	9	8	2	5	1	3	7	6
6	8	7	3	4	9	1	2	5
3	4	1	5	2	8	9	6	7
5	2	9	6	1	7	8	3	4

(b)

Figure 3. Exemplo de Sudoku (9×9) classificado com o nível de dificuldade Difícil. (a) Instância parcialmente preenchida. (b) Solução.

3.2. Saída (Pós-condições)

As soluções propostas retornam dois possíveis valores de saída: **verdadeiro** ou **falso**. A solução ser considerada verdadeira implica que o sudoku teve todas as suas células preenchidas, seguinte rigorosamente as restrições definidas anteriormente, garantindo também que há somente uma forma possível de resposta. Nos casos em que o retorno é falso, significa que o algoritmo não foi possível de resolver o quebra-cabeça proposto.

4. Backtracking

4.1. Funcionamento do algoritmo

- O algoritmo recebe uma matriz $(n \times n)$ que representa a grid e sua ordem (N) .
- É verificado se na grid há presença de células vazias.

- Se sim, é atribuído a essa célula números de 1 a N , e para cada número atribuído, é analisado se ao colocar esse número na célula, a grid segue sendo segura, ou seja, não possui números repetidos na mesma linha, coluna e bloco daquela célula. Sendo segura, o algoritmo é chamado recursivamente e o processo é repetido para a próxima célula. Caso aquele número torne a grid insegura, ela é limpa e é feita uma nova tentativa por meio de *backtracking*.
- No caso de todos os números testados tornarem a grid insegura, o algoritmo não é capaz de resolver o sudoku, retornando falso.
- As chamadas recursivas em que o retorno é verdadeiro, o loop vai se resolvendo recursivamente.
- O algoritmo é encerrado quando é verificado que não há mais células vazias na grid, significando que o sudoku está resolvido, retornando verdade.

4.2. Pseudocódigo

O pseudocódigo do Backtracking pode ser encontrado no código descrito no algoritmo 1. A função auxiliar *IsSafe* se encontra em algoritmo 2.

Algorithm 1 Sudoku Backtracking Solver

```

1: procedure SOLVESUDOKU(grid, row, col)
2:   Input: grid (Sudoku grid), row (current row), col (current column)
3:   if row = degree - 1 and col = degree then
4:     return true
5:   end if
6:   if col = degree then
7:     row  $\leftarrow$  row + 1
8:     col  $\leftarrow$  0
9:   end if
10:  if grid[row][col]  $\neq$  0 then
11:    return SOLVESUDOKU(grid, row, col + 1)
12:  end if
13:  for num = 1 to degree do
14:    if ISSAFE(grid, row, col, num) then
15:      grid[row][col]  $\leftarrow$  num
16:      if SOLVESUDOKU(grid, row, col + 1) then
17:        return true
18:      end if
19:      grid[row][col]  $\leftarrow$  0
20:    end if
21:  end for
22:  return false
23: end procedure

```

Algorithm 2 Sudoku Backtracking Solver

```
1: procedure ISSAFE(grid, row, col, num)
2:   Input: grid (Sudoku grid), row (current row), col (current column), num (number
      to place)
3:   for x = 0 to degree do
4:     if grid[row][x] = num then
5:       return false
6:     end if
7:   end for
8:   for x = 0 to degree do
9:     if grid[x][col] = num then
10:      return false
11:    end if
12:  end for
13:  startRow  $\leftarrow$  row - (row % kRow)
14:  startCol  $\leftarrow$  col - (col % kCol)
15:  for i = 0 to kRow do
16:    for j = 0 to kCol do
17:      if grid[i + startRow][j + startCol] = num then
18:        return false
19:      end if
20:    end for
21:  end for
22:  return true
23: end procedure
```

4.3. Análise de complexidade

- **IsSafe:** O método **IsSafe** localizado no Algoritmo 2 realiza uma verificação ao longo de toda a **linha**, toda a **coluna** e de todo o **bloco** que uma **célula** está presente. Logo, podemos inferir que sua complexidade de tempo pode ser dada por $O(3n) = O(n)$.
- **SolveSudoku**
 - **Linhas 3-9:** Essas linhas lidam com verificações e incrementos básicos, como verificar se a solução está completa ou mover para a próxima linha e coluna. Cada uma dessas operações ocorre em tempo constante, portanto, podemos considerar que essa parte do código tem complexidade de $O(1)$.
 - **Linha 10-12:** A linha mais interna deste *if* será executada um número de vezes igual à quantidade de **células** vazias na **grid**.
 - **Linhas 13-21:** Este *for* é responsável por testar todos os possíveis valores para uma célula vazia. Para cada valor, o método *IsSafe* é chamado para verificar se o número pode ser colocado na célula conforme as regras do Sudoku. Se o número for válido, a função chama recursivamente *solveSudoku* para tentar resolver o restante da grade. Caso contrário, o número é removido (backtracking) e o próximo valor é testado.
No pior caso, cada célula precisa testar até n números (onde n é o tamanho do lado da grade, como 9 em um Sudoku 9×9). A recursão tenta preencher

todas as células da grade. Se assumirmos um cenário extremo onde a grade está completamente vazia, cada célula pode ter até n opções.

Portanto, a complexidade temporal no pior caso pode ser estimada como $O(n)$ para uma única célula. Para uma grade completa com n^2 células, considerando os testes recursivos que nos levam para uma complexidade exponencial, o limite superior da complexidade se torna $O(n^{n \times n})$ ou $O(n^{n^2})$. Essa é uma estimativa teórica do pior caso, já que, na prática, muitas tentativas são descartadas rapidamente pelo método *IsSafe*.

4.4. Prova de corretude

Afirmção: O algoritmo de backtracking preenche todas as **células** vazias de um sudoku corretamente, desde que a entrada seja um **sudoku bem formado**¹.

Base da indução:

1. Para o primeiro caso base, consideremos que todas as **células** da entrada estão preenchidas. Nesse caso, significa que temos uma **grid completa** válida. Logo, por definição, temos um sudoku resolvido. Com isso, a busca na *linha 10* do método de resolução será chamada recursivamente para todas as células presentes, até que todas sejam percorridas e a **linha** seja igual ao tamanho $N \times N$. Logo, o *if* da *linha 3* será acionado, finalizando a execução e retornando *true*, garantindo assim que temos uma **grid completa** válida e fornecendo também uma condição de parada.
2. Para o segundo caso base, consideremos que o algoritmo de backtracking recebe uma **grid incompleta** com uma única **célula** vazia. Nesse caso, sabemos que toda a entrada é um sudoku válido, logo, passível de solução. Com isso em mente, podemos concluir que há um **valor** entre 1 e N que torna a entrada em uma **grid completa** válida. Sabendo disso, o *for* da *linha 13* será responsável por testar todos os possíveis valores para aquela **célula**. Em seguida, na *linha 14*, será feita a checagem da validade daquele valor; assim, caso seja válido, o método será chamado recursivamente para a próxima célula vazia. Neste caso, como não há outras células vazias, cairemos no cenário descrito no caso base 1, provando, assim, o segundo caso base.

Hipótese indutiva: Suponha que, para qualquer entrada com kk **células** vazias de uma **grid incompleta**, a entrada é válida, sendo $k \geq 1$ e kk menor ou igual ao total de células da **grid**, subtraído da quantidade de células pré-preenchidas do quebra-cabeça.

Passo indutivo: Agora, vamos provar que a afirmação $P(k) \Rightarrow P(k+1)$ é verdadeira. Dada a hipótese indutiva, sabemos que temos um **sudoku bem formado**, logo existe apenas uma solução válida. Sabendo também que $k \geq 1$, podemos concluir que, dado o cenário da *linha 10*, só entraremos no *for* quando encontrarmos uma **célula vazia**. Com base nisso, o código prosseguirá para a checagem de todas as possibilidades de preenchimento da célula. Logo, sabendo que, se um número escolhido não levar a uma solução

¹Um Sudoku bem formado possui a característica de ter uma solução única, ou seja, existe apenas uma combinação de números que satisfaz as regras do jogo, com um conjunto mínimo de pistas suficiente para que a solução seja dedutível.

válida, ele será removido (backtracking) e o próximo número será tentado. Eventualmente, por exaustão, a única solução válida será encontrada.

Dessa forma, podemos concluir que o algoritmo proposto consegue resolver o quebra-cabeça do sudoku para qualquer instância de entrada válida.

4.5. Exemplos de execução do algoritmo

Backtracking

ENTRADA SUDOKU INCOMPLETO	SAÍDA SUDOKU COMPLETO
<pre>{ {3, 0, 0, 0}, // Linha 1 {0, 4, 0, 1}, // Linha 2 {0, 2, 0, 0}, // Linha 3 {0, 3, 0, 0} // Linha 4 }</pre>	<pre>{ {3, 1, 2, 4}, {2, 4, 3, 1}, {1, 2, 4, 3}, {4, 3, 1, 2} }</pre>
<pre>{ {0, 3, 0, 0, 0, 0, 0, 0, 0}, // Linha 1 {2, 0, 0, 0, 4, 0, 5, 7, 0}, // Linha 2 {8, 0, 0, 0, 0, 0, 3, 0, 6}, // Linha 3 {0, 0, 0, 0, 2, 8, 6, 3, 0}, // Linha 4 {3, 0, 0, 0, 0, 0, 0, 0, 5}, // Linha 5 {6, 0, 0, 0, 0, 0, 0, 0, 7}, // Linha 6 {0, 0, 0, 0, 0, 0, 0, 0, 0}, // Linha 7 {1, 0, 2, 5, 0, 0, 0, 9, 3}, // Linha 8 {4, 5, 0, 9, 8, 0, 0, 6, 0} // Linha 9 }</pre>	<pre>{ {5, 3, 6, 8, 7, 2, 4, 1, 9}, {2, 9, 1, 6, 4, 3, 5, 7, 8}, {8, 7, 4, 1, 5, 9, 3, 2, 6}, {9, 4, 5, 7, 2, 8, 6, 3, 1}, {3, 1, 7, 4, 9, 6, 2, 8, 5}, {6, 2, 8, 3, 1, 5, 9, 4, 7}, {7, 6, 9, 2, 3, 1, 8, 5, 4}, {1, 8, 2, 5, 6, 4, 7, 9, 3}, {4, 5, 3, 9, 8, 7, 1, 6, 2} }</pre>
<pre>{ {3, 0, 0, 4, 0, 0, 8, 9, 0, 0, 0, 0}, // Linha 1 {0, 6, 1, 10, 0, 0, 0, 0, 5, 0, 9, 4}, // Linha 2 {0, 0, 0, 7, 5, 1, 4, 0, 0, 6, 0, 0}, // Linha 3 {0, 0, 3, 0, 4, 12, 0, 11, 7, 0, 2, 8}, // Linha 4 {0, 7, 0, 8, 6, 0, 0, 0, 3, 0, 4, 11}, // Linha 5 {4, 9, 0, 5, 7, 8, 0, 3, 0, 0, 0, 6}, // Linha 6 {0, 4, 7, 0, 1, 2, 12, 0, 0, 0, 11, 0}, // Linha 7 {1, 0, 5, 6, 11, 0, 0, 0, 0, 0, 7, 12}, // Linha 8 {0, 12, 10, 0, 0, 0, 0, 6, 0, 0, 5, 0}, // Linha 9 {0, 0, 0, 2, 3, 7, 11, 0, 10, 0, 0, 9}, // Linha 10 {0, 0, 0, 12, 0, 10, 1, 0, 0, 0, 0, 5}, // Linha 11 {0, 0, 9, 0, 12, 0, 6, 8, 0, 7, 0, 2} // Linha 12 }</pre>	<pre>{ {3, 5, 2, 4, 10, 6, 8, 9, 1, 11, 12, 7}, {12, 6, 1, 10, 2, 11, 3, 7, 5, 8, 9, 4}, {9, 11, 8, 7, 5, 1, 4, 12, 2, 6, 3, 10}, {6, 10, 3, 1, 4, 12, 5, 11, 7, 9, 2, 8}, {2, 7, 12, 8, 6, 9, 10, 1, 3, 5, 4, 11}, {4, 9, 11, 5, 7, 8, 2, 3, 12, 1, 10, 6}, {8, 4, 7, 9, 1, 2, 12, 5, 6, 10, 11, 3}, {1, 2, 5, 6, 11, 3, 9, 10, 8, 4, 7, 12}, {11, 12, 10, 3, 8, 4, 7, 6, 9, 2, 5, 1}, {5, 1, 6, 2, 3, 7, 11, 4, 10, 12, 8, 9}, {7, 8, 4, 12, 9, 10, 1, 2, 11, 3, 6, 5}, {10, 3, 9, 11, 12, 5, 6, 8, 4, 7, 1, 2} }</pre>

Figure 4. Tabela de exemplos do algoritmo backtracking

5. SaturBFS

5.1. Funcionamento do Algoritmo

- O algoritmo recebe um grafo que representa a grid do sudoku.
- Para a inicialização, é instanciada uma fila vazia para armazenar os vértices que já foram coloridos. O vértice que possui a maior saturação do grafo é definido como o inicial, ele é colorido e adicionado na fila.
- Enquanto a fila não estiver vazia, é feita a seguinte execução: O vértice é retirado da fila, e busca-se o vértice adjacente que possui a maior saturação desse vértice removido.
- Com o adjacente de maior saturação definido, ele é colorido pela função *colorVertex*. Caso a função não consiga encontrar uma cor para esse vértice (não há número válido para aquela célula), o retorno é falso, significando que não é possível resolver o sudoku e o algoritmo é encerrado.

- Se o vértice adjacente for colorido, ele é adicionado na fila e uma nova busca pelo adjacente de maior saturação é feita, repetindo o processo de tentativa de coloração, até que todos os vértices adjacentes sejam coloridos.
- O algoritmo é encerrado quando todos os vértices do grafo forem coloridos com sucesso e quando não há mais vértices na fila para serem analisados, retornando verdade.

5.2. Pseudocódigo

Algorithm 3 Heuristic Saturated Breadth-First Search (SaturBFS)

```

1: Input: Graph  $G$ 
2: Output: True or False
3:  $queue \leftarrow \emptyset$ 
4:  $beginVertex \leftarrow \text{highestSaturationVertex}(G)$ 
5:  $\text{colorVertex}(beginVertex)$ 
6:  $\text{insert}(queue, beginVertex)$ 
7: while  $queue \neq \emptyset$  do
8:    $vertex \leftarrow \text{remove}(queue)$ 
9:    $adjacentVertex \leftarrow \text{highestSaturationAdjacent}(G, vertex)$ 
10:  while  $adjacentVertex \neq \emptyset$  do
11:     $color \leftarrow \text{colorVertex}(adjacentVertex)$ 
12:    if  $color = \emptyset$  then
13:      return False
14:    else
15:       $\text{insert}(queue, adjacentVertex)$ 
16:       $adjacentVertex \leftarrow \text{highestSaturationAdjacent}(G, vertex)$ 
17:    end if
18:  end while
19: end while
20: return True

```

5.3. Análise de complexidade

- **highestSaturationVertex:** Dado o algoritmo utilizar como base o BFS, inicialmente precisamos ter um ponto de origem. No presente algoritmo, começamos pelo vértice de maior saturação, ou seja, aquele que tem a maior quantidade de vértices adjacentes coloridos. Para isso, utilizamos uma função auxiliar que, para cada vértice do grafo, verifica seu grau de saturação percorrendo sua lista de adjacentes. Assim, temos uma complexidade de $O(V * Adj)$.
- **colorVertex:** Para a coloração do vértice, utiliza-se uma função auxiliar que dado um vértice como entrada, para cada cor (No nosso caso, a cor é representada por um número de 1 a K), verifica se ela já foi utilizada na sua lista de adjacentes. Uma vez encontradas as cores disponíveis, o vértice é colorido aleatoriamente com uma delas. Ao final da função, é retornado a cor selecionada aleatoriamente ou nulo caso não encontre uma cor disponível. Logo, temos que essa função tem a complexidade de $O(K * Adj)$.

- **highestSaturationAdjacent:** Assim como o BFS, precisamos percorrer todos os vizinhos (vértices adjacentes) do primeiro vértice da fila, contudo, diferentemente dele, precisamos percorrer esses vértices do mais saturado ao menos saturado. Logo, o algoritmo utiliza uma função auxiliar que dado um grafo e um vértice de origem, verifica o grau de saturação da sua lista de adjacentes, retornando aquele que possui a maior saturação. Dito isso, temos que essa função possui a complexidade de $O(Adj * Adj) = O(Adj^2)$.
- **while externo:** Como todos os vértices do grafo são visitados, temos que a complexidade é $O(V)$.
- **while interno:** Dado que o algoritmo precisa colorir o vértice adjacente recebido e em seguida buscar o próximo adjacente que possua o maior grau de saturação, temos que a complexidade é $O(K * Adj) + O(Adj^2)$.
- **while externo + while interno:** Como dito anteriormente, todos os vértices do grafo serão visitados apenas uma vez, e para cada vértice, será tentado colorir todos os seus vértices adjacentes. Logo temos que a complexidade é $O(V) * [O(Adj^2) + O(K * Adj)]$

Portanto, concluímos que a complexidade do SaturBFS é $O(V * Adj) + O(K * Adj) + O(V) * [O(Adj^2) + O(K * Adj)]$

5.4. Análise do funcionamento

A fim de provar formalmente a corretude do SaturBFS, apresentaremos invariantes de laço e provar a validade de cada umas delas.

Invariante do Laço 1: No início de cada iteração do laço **while**, com *adjacentVertex* $\neq \emptyset$, temos que todos os vértices presentes na *queue* já foram coloridos(preenchidos).

Demonstração. A prova se dá por indução no tamanho da lista de adjacentes do vértice guardado na variável *vertex*.

Caso Básico (Inicialização): No início da primeira iteração do laço **while**, temos que a *queue* não possui nenhum elemento, dado que o vértice inicial já foi colorido e removido. Dito isso, por vacuidade temos que a invariante é válida.

Passo Indutivo (Manutenção): Suponha que o invariante de laço seja verdadeiro no início da iteração onde a lista de adjacentes tem um tamanho k e que entramos no laço **while** para o vértice adjacente mais saturado, com $k > 0$. Vamos provar que o invariante continuará verdadeiro no início da próxima iteração, que será para um tamanho da lista de adjacentes $k = k - 1$. Para isso, vamos executar o corpo do laço **while** descrevendo o que acontece nesta iteração. Na linha 11 é repassado para o método *colorVertex* o vértice adjacente mais saturado atualmente, esse método tentará colori-lo(com uma cor que ainda não foi utilizada na sua lista de adjacentes) retornando a cor utilizada em caso de sucesso ou vazio no caso de falha.

- **Sub-caso 1:** Vamos começar com o caso onde a variável *color* é vazia. Neste caso entramos no *if* da linha 12, uma vez que não foi possível colorir o vértice adjacente mais saturado daquela iteração. Isso significa que o algoritmo não conseguiu resolver o sudoku corretamente, o que faz com o que a execução dele seja encerrada.

- **Sub-caso 2:** $color \neq \emptyset$. Neste caso caímos no *else* da linha 14, uma vez que foi possível colorir o vértice adjacente mais saturado daquela iteração. Dito isso, o algoritmo executará a linha 15, onde o vértice colorido será adicionado a *queue*, seguido da linha 16 que retornará o próximo vértice mais saturado que ainda não foi colorido, caso k ainda seja maior do que 0.

Portanto, a partir dos dois sub-casos analisados, concluímos que a nossa invariante do laço é mantida para uma iteração onde $k = k - 1$, uma vez que apenas adicionamos um vértice na *queue* caso ele seja colorido. Caso contrário, nossa *queue* continuará no mesmo estado (apenas elementos coloridos) que se encontrava no momento que a lista de adjacentes era de tamanho k , pois, a execução terminou.

Término: O laço **while** termina em uma das seguintes ocasiões:

- **Não é possível colorir o vértice adjacente daquela iteração k :** Esse caso foi coberto no **sub-caso 1** da nossa prova indutiva. Logo, podemos concluir que essa finalização manterá a nossa invariante válida.
- **A lista de adjacentes possui todos os vértices coloridos:** Como mostramos no nosso passo indutivo no sub-caso 2, nossa invariante do laço é satisfeita no início de toda iteração, como o teste de saída é a primeira ação da iteração e não há código pós-loop logo nossa invariante continua válida.

Invariante do Laço 2: No início de cada iteração do laço **while**, com $queue \neq \emptyset$, temos que todos os elementos que já foram removidos da *queue* e os que ainda estão presentes já foram coloridos.

Demonstração. A prova se dá por indução na quantidade de vértices que ainda não foram coloridos.

Caso Básico (Inicialização): No início da primeira iteração do laço **while**, temos que nossa *queue* ainda não teve nenhum elemento removido, por vacuidade temos que nossa primeira afirmação é válida. Além disso, temos que *queue* contém um único elemento que já foi colorido na linha 5, logo nossa segunda afirmação é verdadeira, o que também torna a nossa invariante válida.

Passo Indutivo (Manutenção): Suponha que o invariante de laço seja verdadeiro no início de uma iteração onde temos que a quantidade de vértices ainda não coloridos seja k , com $k > 0$ e que entramos no laço **while**, pois $queue \neq \emptyset$. Vamos provar que o invariante continuará verdadeiro no início da próxima iteração, que será quando a quantidade de vértices não coloridos for $k = k - 1$. Para isso, vamos executar o corpo do laço **while** descrevendo o que acontece nesta iteração. Ao executar a linha 8 o algoritmo remove da *queue* o primeiro elemento no qual já está colorido, garantido que não passemos pelo vértice mais de uma vez. Na linha 9 o algoritmo chama o método *highestSaturationAdjacent* que recebe o grafo e o vértice obtido na remoção da linha anterior retornando o vértice adjacente de maior saturação. Pelo o invariante do laço 1 após executar as linhas 10 a 18, temos que todos os vértices presentes na *queue* já foram coloridos (preenchidos) o que garante a invariante no início da próxima iteração.

Término: Sabemos que o algoritmo pode terminar e uma das seguintes opções:

- **Caso 1:** O algoritmo foi interrompido na linha 13, no cenário descrito na prova da invariante 1, no qual afirma que todos os vértices presentes na *queue* já foram

coloridos(preenchidos). Tendo em vista que não a remoção de elementos nesse trecho, podemos concluir que nossa invariante continua verdadeira ao término do laço retornando falso para indicar que não foi possível resolver o sudoku.

- **Caso 2:** Este caso ocorre quando a nossa *queue* = $\emptyset$, ou seja, a fila está vazia, indicando a condição de parada do laço. Para que isso aconteça, todos os vértices não coloridos precisam ser percorridos, coloridos, adicionados à fila e, em seguida, removidos. Esse processo garante que a invariante do algoritmo seja mantida como verdadeira. Assim, o sudoku está resolvido e o algoritmo está correto.

5.5. Exemplos de execução do algoritmo

SaturBFS

ENTRADA SUDOKU INCOMPLETO	SAÍDA SUDOKU COMPLETO
<pre>{ 1, 0, 3, 0, // Linha 1 0, 4, 0, 2, // Linha 2 2, 0, 4, 0, // Linha 3 0, 3, 0, 1 // Linha 4 }</pre>	<pre>{ {1, 2, 3, 4}, {3, 4, 1, 2}, {2, 1, 4, 3}, {4, 3, 2, 1} }</pre>
<pre>{ 0, 3, 0, 0, 0, 0, 0, 0, 0, // Linha 1 2, 0, 0, 0, 4, 0, 5, 7, 0, // Linha 2 8, 0, 0, 0, 0, 0, 3, 0, 6, // Linha 3 0, 0, 0, 0, 2, 8, 6, 3, 0, // Linha 4 3, 0, 0, 0, 0, 0, 0, 0, 5, // Linha 5 6, 0, 0, 0, 0, 0, 0, 0, 7, // Linha 6 0, 0, 0, 0, 0, 0, 0, 0, 0, // Linha 7 1, 0, 2, 5, 0, 0, 0, 9, 3, // Linha 8 4, 5, 0, 9, 8, 0, 0, 6, 0 // Linha 9 }</pre>	<pre>{ {5, 3, 6, 8, 7, 2, 4, 1, 9}, {2, 9, 1, 6, 4, 3, 5, 7, 8}, {8, 7, 4, 1, 5, 9, 3, 2, 6}, {9, 4, 5, 7, 2, 8, 6, 3, 1}, {3, 1, 7, 4, 9, 6, 2, 8, 5}, {6, 2, 8, 3, 1, 5, 9, 4, 7}, {7, 6, 9, 2, 3, 1, 8, 5, 4}, {1, 8, 2, 5, 6, 4, 7, 9, 3}, {4, 5, 3, 9, 8, 7, 1, 6, 2} }</pre>
<pre>{ 3, 0, 0, 4, 0, 0, 8, 9, 0, 0, 0, 0, // Linha 1 0, 6, 1, 10, 0, 0, 0, 0, 5, 0, 9, 4, // Linha 2 0, 0, 0, 7, 5, 1, 4, 0, 0, 6, 0, 0, // Linha 3 0, 0, 3, 0, 4, 12, 0, 11, 7, 0, 2, 8, // Linha 4 0, 7, 0, 8, 6, 0, 0, 0, 3, 0, 4, 11, // Linha 5 4, 9, 0, 5, 7, 8, 0, 3, 0, 0, 0, 6, // Linha 6 0, 4, 7, 0, 1, 2, 12, 0, 0, 0, 11, 0, // Linha 7 1, 0, 5, 6, 11, 0, 0, 0, 0, 0, 7, 12, // Linha 8 0, 12, 10, 0, 0, 0, 0, 6, 0, 0, 5, 0, // Linha 9 0, 0, 0, 2, 3, 7, 11, 0, 10, 0, 0, 9, // Linha 10 0, 0, 0, 12, 0, 10, 1, 0, 0, 0, 0, 5, // Linha 11 0, 0, 9, 0, 12, 0, 6, 8, 0, 7, 0, 2 // Linha 12 }</pre>	<pre>{ {3, 5, 2, 4, 10, 6, 8, 9, 1, 11, 12, 7}, {12, 6, 1, 10, 2, 11, 3, 7, 5, 8, 9, 4}, {9, 11, 8, 7, 5, 1, 4, 12, 2, 6, 3, 10}, {6, 10, 3, 1, 4, 12, 5, 11, 7, 9, 2, 8}, {2, 7, 12, 8, 6, 9, 10, 1, 3, 5, 4, 11}, {4, 9, 11, 5, 7, 8, 2, 3, 12, 1, 10, 6}, {8, 4, 7, 9, 1, 2, 12, 5, 6, 10, 11, 3}, {1, 2, 5, 6, 11, 3, 9, 10, 8, 4, 7, 12}, {11, 12, 10, 3, 8, 4, 7, 6, 9, 2, 5, 1}, {5, 1, 6, 2, 3, 7, 11, 4, 10, 12, 8, 9}, {7, 8, 4, 12, 9, 10, 1, 2, 11, 3, 6, 5}, {10, 3, 9, 11, 12, 5, 6, 8, 4, 7, 1, 2} }</pre>

Figure 5. Tabela de exemplos do algoritmo SaturBFS

6. Resultados

A partir dos estudos realizados, ambos os algoritmos foram implementados e testados com entradas de três níveis de dificuldade em sudokus de (4×4), (6×6), (8×8), (9×9), (10×10) e (12×12). A implementação dos algoritmos pode ser encontrada em nosso repositório no Github: <https://github.com/VictorM-Coder/Sudoku-Problem>. Para o backtracking, o algoritmo foi executado 1000 vezes para cada entrada, de modo a obter uma média mais precisa do tempo de execução. Analogamente, para o SaturBFS, cada entrada foi executada 30.000 vezes, como forma de garantir uma maior precisão na média do tempo de execução e obter uma porcentagem das vezes em que o algoritmo consegue

resolver corretamente as entradas. As especificações do ambiente de execução estão descritas na Tabela 1. Os dados da execução do backtracking estão dispostos na Tabela 2, enquanto os dados da execução do SaturBFS estão na Tabela 3.

Hardware	Especificações
Notebook	Acer Aspire A315-23G
Processador	AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx, 2.10 GHz
RAM instalada	12,0 GB (utilizável: 9,94 GB)
Tipo de sistema	Sistema operacional de 64 bits, processador baseado em x64
Armazenamento	SSD Sata 932 GB
SO	Informações
Edição	Windows 11 Home Single Language
Versão	23H2
Compilação do SO	22631.4169
Ambiente	Detalhes
Programa em execução	Apenas o IntelliJ IDEA com o código a ser executado estava em primeiro plano
Modo de desempenho	”Equilibrado”
Fonte de energia	Ligado à tomada

Table 1. Especificações do ambiente de execução

Dificuldade	Média de Execução (s)	Precisão (%)
Sudoku 4x4		
EASY	0,0000570000000000	100,00
MEDIUM	0,0000250000000000	100,00
HARD	0,0000070000000000	100,00
Sudoku 6x6		
EASY	0,0000310000000000	100,00
MEDIUM	0,0001200000000000	100,00
HARD	0,0000580000000000	100,00
Sudoku 8x8		
EASY	0,0000150000000000	100,00
MEDIUM	0,0007930000000000	100,00
HARD	0,0007670000000000	100,00
Sudoku 9x9		
EASY	0,0000130000000000	100,00
MEDIUM	0,0006490000000000	100,00
HARD	0,0006530000000000	100,00
Sudoku 10x10		
EASY	0,0058969999999999	100,00
MEDIUM	0,0240919999999999	100,00
HARD	0,0301429999999999	100,00
Sudoku 12x12		
EASY	0,0000620000000000	100,00
MEDIUM	0,4089370000000004	100,00
HARD	0,4524310000000008	100,00

Table 2. Desempenho do algoritmo Backtracking para diferentes tamanhos e níveis de dificuldade do Sudoku.

Dificuldade	Média de Execução (s)	Precisão (%)
Sudoku 4x4		
EASY	0,0000184666666667	88,90
MEDIUM	0,0000192000000000	55,19
HARD	0,0000193333333333	54,99
Sudoku 6x6		
EASY	0,0000276000000000	8,94
MEDIUM	0,0000324666666667	0,63
HARD	0,0000313333333333	0,69
Sudoku 8x8		
EASY	0,0000573000000000	0,92
MEDIUM	0,0001120000000000	0,02
HARD	0,0000949000000000	0,04
Sudoku 9x9		
EASY	0,0001298666666667	35,25
MEDIUM	0,0001212333333333	0,00
HARD	0,0001213333333333	0,00
Sudoku 10x10		
EASY	0,0001419666666667	0,03
MEDIUM	0,0001514000000000	0,00
HARD	0,0001499000000000	0,00
Sudoku 12x12		
EASY	0,0001298666666667	0,00
MEDIUM	0,0002483666666667	0,00
HARD	0,0002496000000000	0,00

Table 3. Desempenho do algoritmo SaturBFS para diferentes tamanhos e níveis de dificuldade do Sudoku.

References

- Delahaye, J.-P. (2006). The science behind sudoku. *Scientific American*, 294(6):80–87.
- Horoufiany, M. and Ghandehari, R. (2018). Optimization of the sudoku based reconfiguration technique for pv arrays power enhancement under mutual shading conditions. *Solar Energy*, 159:1037–1046.
- Jana, S., Maji, A. K., and Pal, R. K. (2019). A novel spn-based video steganographic scheme using sudoku puzzle for secured data hiding. *Innovations in Systems and Software Engineering*, 15(1):65–73.
- Li, X., Luo, Y., and Bian, W. (2021). Retracing extended sudoku matrix for high-capacity image steganography. *Multimedia Tools and Applications*, 80(12):18627–18651.
- Lin, H.-H. and Wu, I.-C. (2011). An efficient approach to solving the minimum sudoku problem. *ICGA Journal*, 34:191–208.
- Oliveira, T. A. d. (2024). Sudoku: um estudo sobre algoritmos de geração, classificação e resolução para aplicação educacional.
- Takano, K., de Freitas, R., and de Sá, V. (2015). O jogo de lógica sudoku: modelagem teórica, np-completude e estratégias algorítmicas exatas e heurísticas. In *Anais do XXXIV Concurso de Trabalhos de Iniciação Científica da SBC*, pages 71–80, Porto Alegre, RS, Brasil. SBC.
- Wang, C., Sun, B., Du, K.-J., Li, J.-Y., Zhan, Z.-H., Jeon, S.-W., Wang, H., and Zhang, J. (2024). A novel evolutionary algorithm with column and sub-block local search for sudoku puzzles. *IEEE Transactions on Games*, 16(1):162–172.
- Yato, T. and Seta, T. (2003). Complexity and completeness of finding another solution and its application to puzzles. *IEICE transactions on fundamentals of electronics, communications and computer sciences*, 86(5):1052–1060.